

Load Balancing Scheduling for Batch-Ordered Job-Store: Online vs. Offline

Mengbing Zhou , Yang Wang , *Member, IEEE*, Bocong Zhao , *Graduate Student Member, IEEE*, and Chengzhong Xu , *Fellow, IEEE*

Abstract—Efficient resource utilization is crucial in real-world applications, especially for balancing loads across machines handling specific job types. This paper introduces a novel batch-ordered job-store scheduling model, where jobs in a batch are scheduled sequentially, with their operations allocated in a round-robin fashion across two scenarios. We establish that this problem is NP-hard and analyze it in both online and offline settings. In the online case, we first examine the exclusive scenario, where operations within the same job must be scheduled on different machines, and show that a load greedy (LG) algorithm achieves a tight competitive ratio of $2 - \frac{1}{m}$, with m representing the number of machines. Next, we consider the circular scenario, which requires maintaining the circular order of operations across ordered machines. In this context, we analyze potential anomalies in load distribution during local optimality achieved by the ordered load greedy (OLG) algorithm and provide bounds on the occurrence of these anomalies and the maximum load in each local scheduling round. In the offline case, we abstract each OLG scheduling process as a generalized circular sequence alignment (CSA) problem and develop a dynamic programming-based matching (DPM) algorithm to solve it. To further enhance load balancing, we develop a dynamic programming-based optimization (DPO) algorithm to schedule multiple jobs simultaneously in both scenarios. Experimental results confirm the efficiency of DPM for the CSA problem, and we validate the load balancing effectiveness of both online and offline algorithms using real traffic datasets. These theoretical findings and algorithmic implementations lay a solid groundwork for future practical advancements.

Index Terms—Job-store, round-robin, load balancing, competitive ratio, circular sequence alignment.

Received 9 December 2024; revised 28 June 2025; accepted 23 August 2025. Date of publication 2 September 2025; date of current version 10 October 2025. Recommended for acceptance by Y. Chen. This work was supported in part by the 3rd Xinjiang Scientific Expedition Program under Grant 2021xjkk1300, in part by the Shenzhen Science and Technology Plan Project (Shenzhen-Hong Kong-Macau Category C) under Grant SGDX20220530111001003, and in part by the Shenzhen Science and Technology Program under Grant CJGJZD20230724093659004. (*Corresponding author: Yang Wang.*)

Mengbing Zhou and Bocong Zhao are with Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen 518055, China, and also with the University of Chinese Academy of Sciences, Beijing 1000493, China.

Yang Wang is with Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen 518055, China, and also with the University of Chinese Academy of Sciences, Beijing 1000493, China, and also with Shenzhen University of Advanced Technology, Shenzhen 5181074, China (e-mail: yang.wang1@siat.ac.cn).

Chengzhong Xu is with the Faculty of Science and Technology, University of Macau, Macau, China.

Digital Object Identifier 10.1109/TC.2025.3603725

I. INTRODUCTION

THE *job-store scheduling problem* is a well defined problem to model various practical real-world problems [1], [2], [3], [4]. In the job-store, a job in batch consists of a sequence of operations that could be scheduled in a round-robin fashion onto a set of machines. Superficially, it resembles the classic job-shop problem [5], but they are quite different in nature. Unlike job-shop scheduling, where at each step operations must be assigned to their *designated* machines, job-store scheduling allows them to be scheduled on *any* available machine. Moreover, job-shop scheduling typically requires each operation to start only after its immediate predecessor has completed. In contrast, job-store scheduling distributes operations in a round-robin manner, subject to certain structural constraints, without strictly considering execution order. This model aligns with the concepts of concurrency and batch processing [6], [7]. Specifically, on the same machine, multiple tasks can co-exist simultaneously. The machine automatically selects executable tasks based on their status and processes them, while temporarily suspending tasks that cannot be executed. Under multi-tasking conditions, this method neither leads to resource wastage nor results in overall performance degradation. Thus, the job-store problem is inherently distinct from the job-shop problem, as well as from parallel machine scheduling [8], flow shop scheduling [9], and other existing scheduling models [10], [11].

To the best of our knowledge, our scheduling problem is novel and presents a research scenario that has not been extensively explored, with no dedicated algorithm design or theoretical analysis available. In light of this situation, the research objective of this paper is to establish a theoretical foundation for the job-store problem. By abstracting key characteristics from practical scheduling challenges, we aim to analyze the complexity and approximation possibilities of this problem from an algorithmic perspective. To this end, we assume that the jobs are organized in an ordered batch, each pipelining a sequence of operations with varying durations. We refer to the problem as the *batch-ordered job-store scheduling problem* when considering load balancing across the machines.

A. Motivation

We can identify such scheduling problems in many practical applications [1], [2], [3], [4]. In parallel computing, different

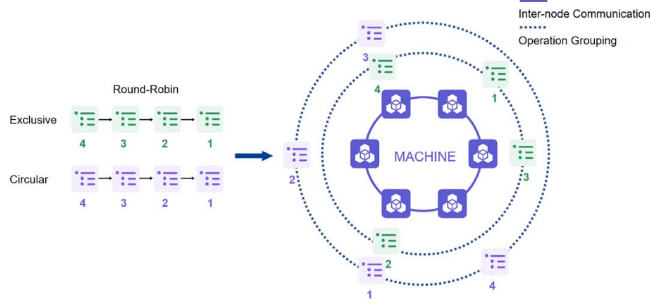


Fig. 1. Job-store scheduling model. The green job illustrates the exclusive scenario, requiring operations to be placed on different machines. The purple job illustrates the circular scenario, where operations must follow the circular order of machines during allocation.

stages of a pipeline often need to be executed on separate GPU or processors to avoid contention and ensure efficient execution [1]. In wireless communication systems, tasks occupying overlapping frequency bands must be assigned to different channels to prevent signal interference [12]. Other examples include the scheduling of multiprocessor tasks with chain precedence on large multiprocessor systems [2], and I/O operations in parallel file systems, where files are typically divided into stripes and distributed across multiple disks in a round-robin manner [3].

Among these applications, the pipeline-on-ring architecture represents a particularly structured case. In supercomputing, for example, compute nodes are arranged in a closed loop, with each node responsible for one stage of the pipeline and communicating only with its immediate neighbors. Tasks are passed sequentially along the ring, enabling ordered execution while reducing communication overhead. This design emphasizes the importance of maintaining both stage order and balanced workload distribution across compute nodes [13].

Motivated by these real-world applications, we study two specific scenarios in this paper, each following different structural constraints during the round-robin scheduling process:

- 1) **Exclusive Scenario:** No two or more operations from the same job are assigned to the same machine during round-robin scheduling.
- 2) **Circular Scenario:** Operations from the same job are scheduled in a round-robin fashion, with machines selected in a cyclic order.

These two scenarios are chosen as representative settings for job-store scheduling, capturing key constraints commonly encountered in real-world scheduling applications. The exclusive scenario reflects situations where operations in a job must be processed on different machines to avoid resource contention or interference [1]. The circular scenario, on the other hand, captures order-preserving constraints in a circular deployment, such as those found in rotary systems or ring-based data centers, where the spatial order of subtask allocation impacts overall efficiency and correctness [14]. These two scheduling scenarios are shown in Fig. 1.

Building on these representative cases, we further distinguish between online and offline scheduling modes. In many real-world systems, tasks may arrive dynamically without prior

knowledge (online) [15], while in others, task batches may be known in advance or temporarily buffered (offline) [16]. To cover both types of systems, we consider both online and offline scheduling settings, each representing a common and fundamental situation in practice.

While numerous studies have explored various scheduling problems, each with its own constraints, focus, and methodologies [17], [18], [19], [20], these approaches are not fully applicable to our problem due to its unique nature. Although general methods, such as heuristic or learning-based approaches, could be designed to tackle this issue, they fail to provide explicit performance guarantees and cannot prevent extreme degradation in certain cases. Therefore, there is a strong demand for rule-based algorithms supported by robust theoretical analysis tailored to these specific scenarios. Thus, our work investigates both the exclusive and circular scenarios under online and offline settings to lay the foundation for this line of scheduling research.

B. Contributions

In the online case, we propose local greedy algorithms to schedule batch-ordered jobs, aiming for locally optimal scheduling for each job. For the exclusive scenario, we derive several lemmas and establish a tight competitive ratio. For the circular scenario, we investigate potential load distribution anomalies that arise as local optimality is achieved, establishing bounds on the number of anomalies and the load for each scheduling.

In the offline case, we first treat each job as a sequence and design optimal algorithms for aligning two sequences. Specifically, we extract core steps from the online algorithm to align sequences in the offline exclusive scenario. For the circular scenario, we formulate a generalized Circular Sequence Alignment (CSA) problem (covering both online and offline cases) and propose an efficient alignment algorithm. Following this, we consider scheduling multiple jobs simultaneously, determining their optimal combination order to enhance load balancing in both scenarios. The main research contents of this paper are presented in Table I.

In summary, our key contributions include the following:

- 1) We introduce the batch-ordered job-store scheduling problem and prove that the problem is NP-hard.
- 2) In the online case, we analyze a greedy algorithm that achieves local optimality for the exclusive scenario and demonstrate that the online algorithm attains a tight competitive ratio of $2 - \frac{1}{m}$. We investigate a similar greedy algorithm for the circular scenario, identifying potential anomalies and providing bounds on the number of affected machines as well as the maximum load for each scheduling.
- 3) In the offline case, we model the local scheduling problem in the circular scenario as a CSA problem and develop a Dynamic Programming-based Matching (DPM) algorithm to solve it. Additionally, we introduce a Dynamic Programming-based Optimization (DPO) algorithm to enhance offline scheduling across both scenarios.

TABLE I
SUMMARY OF MAIN RESEARCH CONTENTS

	Online	Offline
Exclusive	LG algorithm, tight competitive ratio $2 - \frac{1}{m}$	func (extracted from LG) + DPO algorithm
Circular	OLG algorithm, maintaining order anomaly, bounds on anomalies and local load	CSA problem, DPM + DPO algorithm

TABLE II
MAIN NOTATIONS IN THIS STUDY

Notation	Definition
m	Number of machines
$\mathcal{B}$	A batch of jobs
J	A job
o_i	The i -th operation load of a job
$o_{\max}$	Maximum operation load so far
S	Sum of loads for all operations so far
τ	Moment when job appears
$\vec{L}$	Load of the machines
L_i	Load on the i -th machine
$L_{\max}$	Maximum load for algorithm
L^*	Maximum load for optimal distribution scheme
ε	A number approaching 0

- 4) We evaluate the performance of the DPM algorithm, including its time and space complexity. Using real-world highway vehicle datasets [21], we assess the load balancing effectiveness of the proposed online and offline algorithms.

Overall, our contributions advance the understanding of batch-ordered job-store scheduling by offering both theoretical insights and practical solutions.

The remainder of this paper is structured as follows. Section II formulates the problem and analyzes its complexity. Section III analyzes online scheduling algorithms and their performance bounds. Section IV presents the implementation details of offline algorithms. Experimental results demonstrating algorithm effectiveness are shown in Section V. Section VI reviews related scheduling problems. Finally, Section VII concludes the paper.

II. PROBLEM FORMULATION

This section presents the system model, detailing two scheduling scenarios and providing an introduction to the proof of NP-hardness. Table II presents the main notations.

A. System Model

The batch-ordered job-store scheduling problem can be formulated as follows: A batch of jobs, denoted by $\mathcal{B} \triangleq \{J_1, J_2, \dots, J_n\}$, needs to be scheduled on m machines. The load on each machine is represented by the vector $\vec{L} \triangleq [L_1, L_2, \dots, L_m] \in \mathbb{R}^{1 \times m}$. Each job $J \in \mathcal{B}$ consists of a set of operations, $J \triangleq \{o_1, o_2, \dots\}$, where the number of operations is between 1 and m . Each job is scheduled in an orderly sequence, and let τ denote the moment at which a job

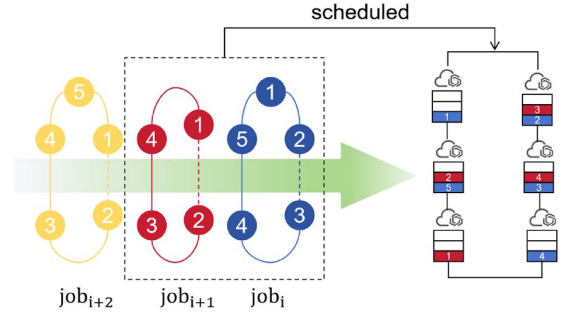


Fig. 2. Circular scenario of the batch-ordered job-store scheduling problem, where both jobs and machines are ordered, and the circular order of operations is preserved after assignment.

appears for scheduling. The objective for the scheduling problem is to achieve load balancing by minimizing the maximum load among all machines. This optimization goal can be formulated as:

$$\min \max \{L_i \mid i \in \{1, 2, \dots, m\}\}.$$

In the scheduling process, two scenarios are investigated: the exclusion scenario and the circular scenario. The scenarios are described as follows:

- 1) **Exclusive scenario:** When assigning job operations to machines in a round-robin manner, operations within each job J must be allocated to different machines. Specifically, for any two operations $o, o' \in J$, if o is assigned to machine L_i and o' is assigned to machine L_j , where $i, j \in \{1, 2, \dots, m\}$, then $i \neq j$ must hold.
- 2) **Circular scenario:** The operations of jobs and machines are both ordered. During the round-robin assignment of operations to machines, the circular order between adjacent operations is preserved, as shown in Fig. 2. Specifically, for any operations $o_p, o_{(p+1)\%m} \in J$, where $p \in \{1, 2, \dots, m\}$, if o_p is assigned to machine L_i and $o_{(p+1)\%m}$ is assigned to machine L_j :
 - If $i < j$, no operation should be assigned to any machine L_q where $q \in \{i+1, \dots, j-1\}$.
 - If $i > j$, no operation should be assigned to any machine L_q where $q \in \{i+1, \dots, m\} \cup \{1, \dots, j-1\}$.

B. Hardness

We establish the NP-hardness of the batch-ordered job-store scheduling problem. First of all, we consider a typical NP-hard problem: Given a set $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$ of positive

numbers, find a partition, i.e., a subset $\mathcal{A}' \subset \mathcal{A}$, such that the residue

$$E = \left| \sum_{a_j \in \mathcal{A}'} a_j - \sum_{a_j \notin \mathcal{A}'} a_j \right| \quad (1)$$

is minimized.

When the problem is transformed into determining whether there exists a partition with $E = 0$, it becomes the classical number partitioning problem, whose NP-completeness was demonstrated in two successive proofs by Karp [22] and, independently, by Garey and Johnson [23].

Theorem 1: Regardless of the structural constraints imposed on jobs during the round-robin scheduling process, the batch-ordered job-store problem remains NP-hard.

Proof: Let s be the sum of all the numbers in the partition problem. If the sum of one subset is a , then the sum of its complementary subset is $s - a$. Then,

$$\begin{aligned} & \text{Minimize. } E \\ & \Leftrightarrow \text{Minimize. } |a - (s - a)| \\ & \Leftrightarrow \text{Minimize. } \max(a, s - a). \end{aligned}$$

Therefore, the partition problem is equivalent to minimizing the sum of the larger subset. This process can be viewed as scheduling jobs—each consisting of a single operation—on two machines to minimize the larger load. As a result, this NP-hard problem can be regarded as a special case of the general batch-ordered job-store problem. Regardless of the structural constraints that jobs may satisfy during the round-robin scheduling process, the partition problem can be polynomially reduced to the corresponding instance of the batch-ordered job-store problem. Hence, the batch-ordered job-store problem remains NP-hard. $\square$

III. ONLINE ANALYSIS

Given the NP-hard nature of the problem and the need for online algorithms in batch-ordered job-store scheduling, we begin by proposing efficient greedy algorithms. In this section, we analyze the bounds of these greedy algorithms across different scenarios.

A. Exclusive Scenario

In this scenario, our primary focus is on designing an effective allocation scheme. Once the allocation is determined, the operations are then assigned in a round-robin fashion. We introduce a simple Load Greedy (LG) algorithm in Algorithm 1. It schedules a job at each moment, arranging the operations in descending order and then sequentially assigning the operation to the machine that is currently least loaded and has no other operations.

Lemma 1: The load difference across machines generated by Algorithm 1 has a least upper bound $o_{\max}$.

Proof: Suppose that the load difference between any machines at the moment $\tau - 1$ is not greater than $o_{\max}$, that is

$$|L_i - L_j| \leq o_{\max}, \forall i, j \in \{1, 2, \dots, m\}, i \neq j. \quad (2)$$

Algorithm 1 Load Greedy (LG)

Input:

m : Number of machines;
 $\mathcal{B}$: A group of jobs.

Output:

$\vec{L}$: Service load of machines.

```

1:  $\vec{L} \leftarrow [0] * m$ 
2: for all  $B$  in  $\mathcal{B}$  do
3:    $\vec{I} \leftarrow \text{sort\_ascend}(\text{index}(\vec{L}))$ ;
4:    $B \leftarrow \text{sort\_descend}(\text{service\_time}(B))$ ;
5:    $pos \leftarrow 0$ ;
6:   for all  $v$  in  $B$  do
7:      $\vec{L}[\vec{I}[pos]] \leftarrow \vec{L}[\vec{I}[pos]] + v$ ;
8:      $pos \leftarrow pos + 1$ ;
9:   end for
10: end for
    
```

And after scheduling a job at moment τ ,

$$L'_i - L'_j > o_{\max}. \quad (3)$$

Since operations from the same job cannot be assigned to the same machine, each machine has at most one operation from the same job, and Eq. (3) can be expressed as

$$(L_i + o_i) - (L_j + o_j) > o_{\max}. \quad (4)$$

According to Eq. (2), if $L_i = L_j$ or $0 < L_j - L_i \leq o_{\max}$, it can be seen that $o_i - o_j > o_{\max}$, which contradicts $o_{\max}$ being the currently largest operation load. If $0 < L_i - L_j \leq o_{\max}$, then it can be seen that $o_i > o_j$, which contradicts Algorithm 1 that defines the assignment of larger operation load to machine with smaller total load. So there is no load difference greater than $o_{\max}$ after any job scheduling.

The load difference can reach $o_{\max}$ when the loads of all machines are equal at moment $\tau - 1$ and there is a job with only one operation with load $o_{\max}$ at moment τ . $\square$

Lemma 2: The maximum load of the machine generated by Algorithm 1 has a least upper bound:

$$\frac{S - o_{\max}}{m} + o_{\max}.$$

Proof: Without loss of generality, let $L_1 \geq L_2 \geq \dots \geq L_m$ be the loads of each machines and $\Delta L_1, \Delta L_2, \dots, \Delta L_{m-1}, 0$ be the differences between the corresponding loads minus L_m , obviously all of them are greater than or equal to 0, then

$$L_1 = L_m + \Delta L_1 = \frac{S - \Delta L_1 - \sum_{i=2}^{m-1} \Delta L_i}{m} + \Delta L_1, \quad (5a)$$

$$L_1 \leq \frac{S - \Delta L_1}{m} + \Delta L_1. \quad (5b)$$

According to Lemma 1 and $m \geq 1$, it holds that

$$L_1 \leq \frac{S - o_{\max}}{m} + o_{\max}. \quad (6)$$

When the same example as in the proof of Lemma 1 occurs, it is clear that the maximum load can be up to $\frac{S - o_{\max}}{m} + o_{\max}$. $\square$

Theorem 2: The competition ratio of Algorithm 1 is $2 - \frac{1}{m}$.

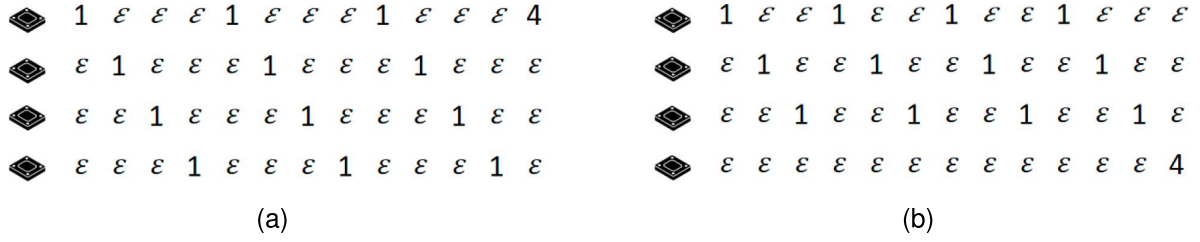


Fig. 3. An example of how the competition ratio is tight where $m = 4$. (a) The result of algorithm 1 scheduling. (b) The result of optimal scheduling.

Proof: According to Lemma 2, we have

$$\frac{L_{\max}}{L^*} \leq \frac{\frac{S-o_{\max}}{m} + o_{\max}}{L^*}. \quad (7)$$

Given the indivisibility of operations, the optimal maximum load, L^* , must be at least as large as the maximum operation load encountered so far, i.e.,

$$L^* \geq o_{\max}. \quad (8)$$

Additionally, since $m \geq 1$, we have the following inequality:

$$\frac{L_{\max}}{L^*} \leq \frac{\frac{S-L^*}{m} + L^*}{L^*} = \frac{1}{L^*} \cdot \frac{S}{m} + \left(1 - \frac{1}{m}\right). \quad (9)$$

According to the *Pigeonhole Principle* [24], at any given moment τ , the optimal maximum load must be at least equal to the average load when the total job loads are distributed equally among the machines, i.e.,

$$L^* \geq \frac{S}{m}, \quad (10)$$

which leads to the following inequality:

$$\frac{L_{\max}}{L^*} \leq \frac{1}{L^*} L^* + \left(1 - \frac{1}{m}\right) = 2 - \frac{1}{m}. \quad (11)$$

Theorem 3: The competition ratio $2 - \frac{1}{m}$ of Algorithm 1 is tight.

Proof: Consider a sequence with $(m-1)m+1$ jobs, where each job has m operations. In the first $(m-1)m$ jobs, the load of one operation is 1, and the load of the other $m-1$ operations is ε ($\varepsilon \rightarrow 0$). In the last job, one operation has a load of m , and the other $m-1$ operations have a load of ε .

Based on Algorithm 1, we can deduce from the allocation pattern that before scheduling the last job, the load of all m machines is $m-1 + (m-1)^2\varepsilon$. Consequently, after executing the algorithm, the maximum load is $2m-1 + (m-1)^2\varepsilon$.

Whereas, in the optimal scheduling scheme, before scheduling the last job, the operations with load 1 are assigned to only $(m-1)$ machines so that they all have load $m + (m-2)m\varepsilon$ and the remaining machine has load $(m-1)m\varepsilon$. Finally, the operations with load m of the last job are assigned to the machine with load $(m-1)m\varepsilon$, and the optimal result is $m + (m-1)m\varepsilon$.

Hence, we can find the competition ratio:

$$\frac{L_{\max}}{L^*} = \lim_{\varepsilon \rightarrow 0} \frac{2m-1 + (m-1)^2\varepsilon}{m + (m-1)m\varepsilon} = 2 - \frac{1}{m}. \quad (12)$$

□

Fig. 3 illustrates an example of Algorithm 1 reaching the competitive ratio. In fact, the example degenerates to the list scheduling problem, i.e., assigning a list of jobs that don't contain operations to different processing units to minimize the maximum load, which has been shown to have the same tight approximation ratio by Graham [25]. Specifically, the list scheduling problem is a special case of the problem we are discussing, so the tight approximation ratio for it is a lower bound for our model, and our proof states it as a greatest lower bound.

Theorem 4: The Algorithm 1 ensures that a locally optimal solution is obtained and minimizes the maximum load difference between the machines at any moment τ .

Proof: Without loss of generality, at moment τ , let $L_m \leq L_{m-1} \leq \dots \leq L_1$ be the load of each machine respectively, and $o_1 \geq o_2 \geq \dots \geq o_m$ be the load of each operation respectively, and both of their tail could be 0. After Algorithm 1 completes the assignment, the load of each machine is $L_m + o_1, L_{m-1} + o_2, \dots, L_1 + o_m$ respectively. Let the maximum value is $L_{m-i+1} + o_i$.

Assuming that there is a combination method with a smaller maximum value, where the $o'_i < o_i$ combine with L_{m-i+1} . Now consider the new position of o_i . Keep the order of $L_m \leq \dots \leq L_{m-i+1} \leq \dots \leq L_1$, if o_i is assigned to the right of L_{m-i+1} , then $L_x + o_i \geq L_{m-i+1} + o_i, x \in \{m-i, \dots, 1\}$, this contradicts the hypothesis. If o_i is assigned to the left of L_{m-i+1} , it's obvious that at least one $o_j \geq o_i, j \in \{1, \dots, i-1\}$ is assigned to the right of L_{m-i+1} , so $L_x + o_j \geq L_{m-i+1} + o_i$, this contradicts the hypothesis. So there is no other combination method with a smaller maximum value than Algorithm 1, i.e., it obtains a locally optimal solution.

Similarly, it can be concluded that no other combination method produces a greater minimum value than Algorithm 1. Therefore, maximum load difference between machines is minimized. □

Algorithm 1 and Theorem 4 discussed above also appear in our previous work [26]; however, the detailed analysis of the competitive ratio presented here is a novel and significant contribution.

B. Circular Scenario

When the scheduling problem goes further, it requires us to maintain the circular left-right order between operations during the round-robin scheduling process. To address this, we adopt an algorithm similar to Algorithm 1, modifying lines 3-9 to achieve locally optimal scheduling that meets the requirements

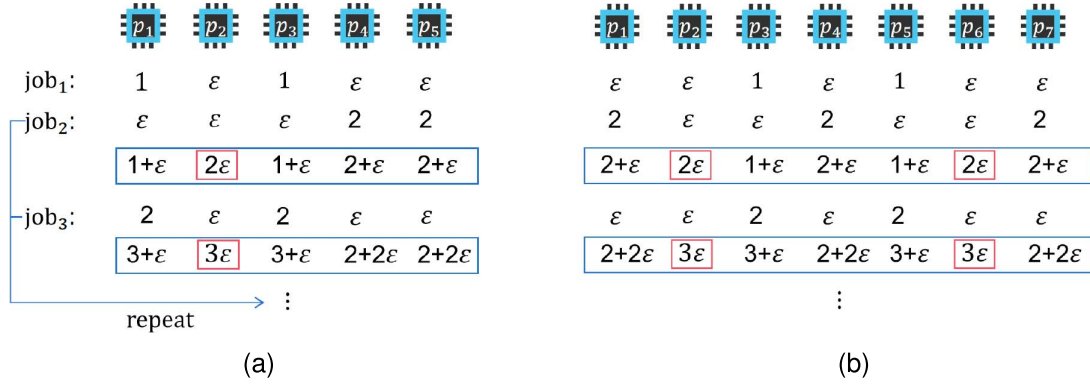


Fig. 4. Examples of maintaining order anomaly. (a) When $m = 5$, one anomaly occurs. (b) When $m = 7$, two anomalies occur.

of the circular scenario. We refer to this modified version as the Ordered Load Greedy (OLG) algorithm. Specifically, when a new job arrives, it is scheduled to the machines in a locally optimal manner, which means that each scheduling minimizes the local maximum load. When there are multiple feasible solutions, one with the smallest load difference is selected. This process is implemented by calling the *optimal_matching_function*, which will be discussed in the next section.

However, we observe that some undesirable situations may occur when we use m machines to handle jobs with up to m operations utilizing such a locally optimal scheduling. As illustrated in Fig. 4, where ε denotes a very small number that can be approximated as 0 when compared to other values, though not exactly zero, the scheduling scheme that minimizes the maximum load is executed each time a job is processed.

We first define a *shift* as the translation of a scheme relative to the current one. A *shift* of k indicates that the current scheme is shifted k positions to the right. We then find that any *shift* relative to each scheme in Fig. 4 would result in a higher load on the machines. That is, each scheme in Fig. 4 is the only locally optimal solution. However, the load on machine highlighted by the red box is significantly lower than that on other machines. If job₂ and job₃ subsequently appear alternately, the load difference between these machines and the other machines can become infinite. For the convenience of the subsequent description, we define this phenomenon as follows:

Definition 1 (Maintaining Order Anomaly (MOA)): At any given moment τ , the machine loads can be categorized into three levels: $L_\alpha > L_\beta \geq L_\lambda$. The job consists of m operations, each having either a large or a small load. During scheduling, there exists a unique optimal assignment such that any shift from this configuration results in a higher maximum load. In this optimal scheme, all L_β machines are assigned the large operations, L_α machines are paired with part of the small operations—thus becoming the new L_α and L_β , respectively—and the remaining L_λ machines are consistently paired with the rest of the small operations.

Further, we investigate how this phenomenon may affect scheduling and determine an upper bound on the number of impacted machines.

Lemma 3: When $f(m) = m - 2\sqrt{m-1}$ is a positive integer, $\sqrt{f(m)}$ must also be an integer.

Proof: Let n be a positive integer, and suppose $m - 2\sqrt{m-1} = n$. By rearranging the equation, we obtain

$$m^2 - (2n+4)m + (n^2+4) = 0. \quad (13)$$

The discriminant of this equation is $16n$. Since m is an integer, $16n$ must be a perfect square, which implies that n must also be a perfect square. Hence, the original proposition is proven. $\square$

Theorem 5: The number of machines exhibiting the Maintaining Order Anomaly has a least upper bound of $m - 2\sqrt{m-1}$.

Proof: Let the number of machines with a load of L_α be α , and the number of machines with a load of L_λ be λ . Then, the number of machines with a load of L_β , which is also the number of large numbers in the job, is $m - \alpha - \lambda$.

For each L_α , there are specific *shifts* that must be avoided to prevent combining with the large numbers in the job. Let the sets of *shifts* that must be avoided relative to the optimal scheme for each L_α be denoted by $\mathcal{A}_1, \mathcal{A}_2, \dots, \mathcal{A}_\alpha$. Since L_α must not be combined with any large value, we have:

$$|\mathcal{A}_i| = m - \alpha - \lambda, \forall i \in \{1, 2, \dots, \alpha\}. \quad (14)$$

Since the optimal scheduling scheme is unique, it follows that

$$\mathcal{A}_1 \cup \mathcal{A}_2 \cup \dots \cup \mathcal{A}_\alpha = \{1, 2, \dots, m-1\}, \quad (15a)$$

$$|\mathcal{A}_1 \cup \mathcal{A}_2 \cup \dots \cup \mathcal{A}_\alpha| = m-1. \quad (15b)$$

Intuitively, the above expression indicates that only the shift of m (the original scheme) does not need to be avoided. Next, we consider the following inequality:

$$m-1 \leq |\mathcal{A}_1| + |\mathcal{A}_2| + \dots + |\mathcal{A}_\alpha| = \alpha(m - \alpha - \lambda). \quad (16)$$

We can deduce that:

$$\lambda \leq m - \left(\alpha + \frac{m-1}{\alpha} \right), \quad (17a)$$

$$\lambda \leq m - 2\sqrt{m-1}. \quad (17b)$$

Additionally, we demonstrate m machines are sufficient to construct $f(m) = m - 2\sqrt{m-1}$ MOAs. In these cases, since $f(m)$ is an integer, according to Lemma 3, $\sqrt{f(m)}$ is also an

integer. A common method to construct $f(m)$ MOAs using m machines is to set job_1 , job_2 , and job_3 in Fig. 4 as follows:

$$\begin{cases} \text{job}_1 \triangleq [[1] + [\varepsilon] * \sqrt{f(m)} * \sqrt{f(m)} + [1] + [\varepsilon] * (\sqrt{f(m)} + 1)] \\ \text{job}_2 \triangleq [\varepsilon] * (m - f(m) - 1) + [2] * (f(m) + 1) \\ \text{job}_3 \triangleq [[2] + [\varepsilon] * \sqrt{f(m)} * \sqrt{f(m)} + [2] + [\varepsilon] * (\sqrt{f(m)} + 1)] \end{cases}.$$

These examples demonstrate that $m - 2\sqrt{m-1}$ serves as the least upper bound for λ . $\square$

Specifically, Fig. 4(a) illustrates a scenario where five machines are both necessary and sufficient to create a single MOA using the common method defined above. In contrast, Fig. 4(b) shows a scenario where seven machines are used to construct two MOAs, which does not follow the common method but still satisfies the established bound.

The occurrence of MOA can be effectively avoided by adopting more complex load balancing metrics such as minimizing the standard deviation of the load across machines locally, or minimizing the sum of the differences between each machine's load and the maximum load. However, we know that the conditions for the occurrence of MOA are extremely stringent, making it very unlikely to happen in actual scheduling processes. Therefore, using the maximum load as a balancing metric remains effective and efficient in practical environments. We have also validated its effectiveness in the subsequent experimental evaluations.

Theorem 6: When using OLG algorithm, at any monment τ , if $m > 1$ and the number of machines where the load difference from the maximum load exceeds $o_{\max}$ is γm , where $0 \leq \gamma < 1$ and γm is an integer, then

$$\frac{L_{\max}}{L^*} \leq 1 + \frac{m-1}{m(1-\gamma)}. \quad (18)$$

Proof: Let S' be the sum of the loads where the load difference from the maximum load does not exceed $o_{\max}$.

For the other $m - \gamma m$ machines, we apply a proof process analogous to Lemma 2, which leads to the following result:

$$L_{\max} \leq \frac{S' - o_{\max}}{m - \gamma m} + o_{\max}. \quad (19)$$

According to $S' \leq S$,

$$\frac{L_{\max}}{L^*} \leq \frac{\frac{S - o_{\max}}{m - \gamma m} + o_{\max}}{L^*}. \quad (20)$$

According to Eq. (8) and $m - \gamma m \geq 1$,

$$\frac{L_{\max}}{L^*} \leq \frac{\frac{S - L^*}{m - \gamma m} + L^*}{L^*}. \quad (21)$$

According to Eq. (10),

$$\frac{L_{\max}}{L^*} \leq 1 + \frac{1}{1-\gamma} - \frac{1}{(1-\gamma)m} = 1 + \frac{m-1}{m(1-\gamma)}. \quad (22)$$

$\square$

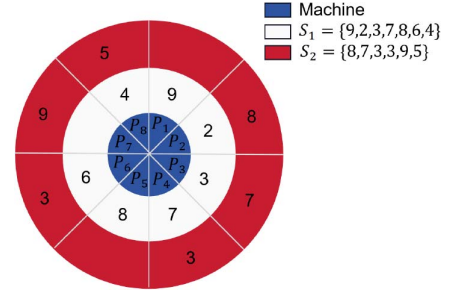


Fig. 5. An example of aligning two sequences, each with a length less than m , into m positions to minimize the sum, where $m = 8$.

IV. OFFLINE IMPLEMENTATION

In this section, we first discuss the implementation of the *optimal_matching_function* in the OLG algorithm, proposing a more generalized circular sequence alignment problem that works for both online and offline situations and solving it. Then, we introduce an optimization method for scheduling multiple jobs simultaneously in an offline setting.

A. Circular Sequence Alignment

In each job scheduling of OLG algorithm, there are two sequences: one sequence of operations and another sequence consisting of machine loads. Generally, we view the process of assigning operations to machines to minimize the maximum load as a circular sequence alignment process.

Furthermore, in the implementation of the alignment method, we consider a more generalized and complex alignment scenario where the lengths of both sequences may be less than m , and the positions of the elements are not yet fixed. This scenario not only addresses the online scheduling case, where one sequence already contains m elements with fixed positions (i.e., machine loads), but also applies to offline scheduling, where two sequences, possibly containing fewer than m elements, need to be aligned, and neither sequence has fixed elements (i.e., two jobs with fewer than m operations that are not yet assigned to specific machines). Fig. 5 illustrates the alignment of two sequences with fewer than m elements and we define this problem as follows:

Definition 2 (Circular Sequence Alignment (CSA)): Given two sequences $S_1 = \{a_1, a_2, \dots, a_{l_1}\}$ and $S_2 = \{b_1, b_2, \dots, b_{l_2}\}$, where $l_1, l_2 \leq m$, initialize two arrays $X = \{x_i = 0 \mid i \in \{1, 2, \dots, m\}\}$ and $Y = \{y_i = 0 \mid i \in \{1, 2, \dots, m\}\}$. Then, replace the elements of X with all elements of S_1 and replace the elements of Y with all elements of S_2 , preserving the circular left-right order of elements in each sequence, such that the objective

$$\min \max \{x_i + y_i \mid i \in \{1, 2, \dots, m\}\}$$

is achieved.

Particularly, it is emphasized that the scheduling constraint in circular scenario defined in Section II is followed, ensuring that the circular order of elements in S_1 and S_2 remains unchanged. Since the solution to this alignment problem may not be unique,

we further refine the approach by selecting the solution that minimizes the maximum difference among the m sums. If multiple solutions still exist, the first valid solution encountered is chosen to ensure efficient load balancing.

To solve the CSA problem, we propose a Dynamic Programming-based Matching (DPM) algorithm. This method achieves circular alignment by sequentially treating each element of the second sequence S_2 as the starting point, splitting S_2 into two parts and swapping their positions, thereby reducing the problem to repeated linear alignment.

In Fig. 5, some positions are not assigned sequence elements, or as stated in Definition 2, the corresponding x or y at these positions remains 0. We refer to these as gaps in the alignment process. For the linear sequence alignment, we initialize a dynamic programming array D , which is a four-dimensional array where $D[i, j, p, q]$ records the current maximum sum of $x + y$, indicating the state of entering the i -th element of sequence S_1 (denoted as a_i) and the j -th element of sequence S_2 (denoted as b_j), with p gaps used in X and q gaps used in Y . The transition process is as follows:

$$D[i, j, p, q] = \min \begin{cases} \max \{D[i-1, j, p, q-1], a_i + 0\} \\ \max \{D[i, j-1, p-1, q], 0 + b_j\} \\ \max \{D[i-1, j-1, p, q], a_i + b_j\} \end{cases} \quad (23)$$

Let l_1 and l_2 represent the lengths of sequences S_1 and S_2 , respectively. The parameters i, j, p, q in Eq. (23) are not independent and must satisfy the following constraints:

$$\begin{cases} 0 \leq i \leq l_1 \\ 0 \leq p \leq m - l_1 \\ 0 \leq j \leq l_2 \\ 0 \leq q \leq m - l_2 \\ i + p = j + q \end{cases} \quad (24)$$

The inequalities in Eq. (24) indicate the index range of the element or gap, and the equation indicates that the elements or gaps in X and Y correspond to each other.

At the beginning of Algorithm 2, we first calculate the boundary conditions i.e. the base cases of the dynamic programming:

$$\begin{cases} D[0, 0, \ell, \ell] = 0 \\ D[\ell, 0, 0, \ell] = \max \{D[\ell-1, 0, 0, \ell-1], a_{\ell-1}\} \\ D[0, \ell, \ell, 0] = \max \{D[0, \ell-1, \ell-1, 0], b_{\ell-1}\} \\ D[\ell, \ell, 0, 0] = \max \{D[\ell-1, \ell-1, 0, 0], a_{\ell-1} + b_{\ell-1}\} \end{cases} \quad (25)$$

The first equation in Eq. (25) represents that both S_1 and S_2 use only gaps, the second and third equations represent that one of S_1 and S_2 uses only gaps and the other uses only elements,

Algorithm 2 DP-based Matching (DPM)

Input:

m : Number of machines;
 S_1, S_2 : Two sequences;

Output:

S_3 : Final sequence after merging.

```

1:  $l_1, l_2 \leftarrow \text{len}(S_1), \text{len}(S_2)$ 
2: if  $m > l_1 + l_2$  then
3:    $S_3 \leftarrow S_1 + S_2$ ;
4:   return;
5: end if
6:  $e_1, e_2 \leftarrow (m - l_1), (m - l_2)$ ;
7: for  $pos \leftarrow 0$  to  $l_2 - 1$  do
8:   Set the index  $pos$  in  $S_2$  as the starting point;
9:   Initialize a  $(l_1 + 1) \times (l_2 + 1) \times (e_1 + 1) \times (e_2 + 1)$  array
      $D$  with all elements set to MAX_VALUE;
10:  Compute all base cases;
11:  for  $i$  in 1 to  $l_1$  do
12:    for  $j$  in 1 to  $l_2$  do
13:      if  $i = j$  then
14:        Loop  $x$  to compute all  $D[i, j, x, x]$  when  $1 \leq x \leq e_1$ ;
15:      else if  $i > j$  then
16:        Loop  $x$  to compute all  $D[i, j, x, x + i - j]$  when
            $0 \leq x + i - j \leq e_2$ ;
17:      else
18:        Loop  $x$  to compute all  $D[i, j, x + j - i, x]$  when
            $0 \leq x + j - i \leq e_1$ ;
19:      end if
20:    end for
21:  end for
22: end for
23: Backtrack through  $D$  to construct  $S_3$ ;
    
```

and the fourth equation represents that both S_1 and S_2 use only elements. The parameter ℓ in Eq. (25) is all possible values that satisfy constraints in Eq. (24).

Theorem 7: The time and space complexity of Algorithm 2 is $O(m^4)$.

Proof: For the time complexity, the outer loop, which selects different elements of S_2 as the starting point, runs in $O(m)$. The inner loop requires $O(m^2)$ time to compute the base cases and backtracking array D , while all the transition processes have a complexity of $O(m^3)$. Combining these factors, the overall time complexity is $O(m^4)$.

Regarding space complexity, the main memory usage comes from the four-dimensional array D . According to Eq. (24), the size of D is approximately $l_1(m - l_1)l_2(m - l_2)$, leading to the following bound:

$$l_1(m - l_1)l_2(m - l_2) \leq \left(\frac{m}{2}\right)^2 \left(\frac{m}{2}\right)^2 = \frac{m^4}{16}. \quad (26)$$

The equality holds when $l_1 = (m - l_1)$ and $l_2 = (m - l_2)$. Consequently, the space complexity is $O(m^4)$. $\square$

By developing an exact algorithm for the CSA problem, we establish that the problem is solvable in polynomial time, which is of considerable theoretical significance. In practice, the proposed algorithm is effective for instances of moderate scale and applicable to a variety of realistic scenarios. However, for large-scale problems, the computational overhead of $O(m^4)$ may hinder its practical deployment. In such cases, approximate

algorithms that balance solution quality and computational efficiency would be more appropriate. Nevertheless, the primary objective of this work is to investigate the structural properties of locally optimal allocation strategies and to identify potential anomalies that may arise during the allocation process. These insights aim to provide a deeper theoretical understanding of batch-ordered job-store scheduling and serve as a foundation for future research. Therefore, the design of efficient approximation algorithms for the CSA problem is beyond the scope of the present study.

B. Multi-Job Optimization

To further enhance load balancing in batch-ordered job-store scheduling, we introduced additional measures inspired by matrix chain multiplication problem. We developed a Dynamic Programming-based Optimization (DPO) algorithm that monitors job conditions over a specified range and schedules multiple consecutive jobs simultaneously. By treating the existing load on the machines as a preceding job and combining it with subsequent jobs into a job sequence called *slice*, we can identify an optimal multi-job merging scheme. This approach enables more effective load balancing in an offline setting and was also applied in our previous work [26].

Assuming that k jobs are scheduled simultaneously each time, the length of the *slice* is $k + 1$. We extract the core steps of Algorithm 1, treating a job as existing machine load and greedily scheduling another job, thereby obtaining an offline algorithm for merging two jobs. This algorithm and the DPM algorithm are used to achieve locally optimal merging of adjacent jobs in different scheduling scenarios, and they are abbreviated as *func* in Algorithm 3.

We use a dynamic programming array d to record the optimal results of merging jobs from the i -th to the j -th job within the *slice*. The base case for this dynamic process occurs when $i = j$:

$$d[i, j] = \text{slice}[i]. \quad (27)$$

During the initialization phase, d is initialized as a three-dimensional array, shaped like an upper triangular matrix, where each element is an array:

$$d = \begin{bmatrix} \text{slice}[0] & [*] & \dots & [*] \\ & \text{slice}[1] & \dots & [*] \\ & & \ddots & \vdots \\ & & & \text{slice}[k] \end{bmatrix}. \quad (28)$$

In the above formula, $'*'$ represents the system's maximum value. For $i < j$, the transition process is as follows:

$$d[i, j] = \min_{i \leq p \leq j-1} \max\{ \text{func}(m, d[i, p], d[p+1, j]) \}, \quad (29)$$

where *func* merges the jobs represented by $d[i, p]$ and $d[p+1, j]$ across m positions.

The final value of d is computed using three nested loops in lines 10-19 of Algorithm 3. These loops operate as follows: the outer loop iterates over the range of merging jobs, the middle loop selects the various starting points within that range, and

Algorithm 3 DP-based Optimization (DPO)

Input:

m : Number of machines;
 k : The number of jobs dispatched at once;
 $\mathcal{B}$: A group of jobs.

Output:

$\vec{L}$: Service load of machines.

```

1:  $\vec{L} \leftarrow [0] * m$ ;
2: while  $\mathcal{B}$  do
3:    $\text{slice} \leftarrow [\vec{L}, \mathcal{B}[0], \dots, \mathcal{B}[k-1]]$ ;
4:    $\mathcal{B} \leftarrow \mathcal{B}[k : \text{end}]$ ;
5:   for  $j \leftarrow 0$  to  $k$  do
6:      $a \leftarrow [\text{MAX\_VALUE}] \times (k - j + 1)$ ;
7:      $d.\text{append}(a)$ ;
8:      $d[j, 0] \leftarrow \text{slice}[j]$ ;
9:   end for
10:  for  $r \leftarrow 1$  to  $k$  do
11:    for  $s \leftarrow 0$  to  $k - r$  do
12:      for  $t \leftarrow s$  to  $s + r - 1$  do
13:         $\text{temp} \leftarrow \text{func}(m, d[s, t-s], d[t+1, s+r-1])$ ;
14:        if  $\max(d[s, r]) > \max(\text{temp})$  then
15:           $d[s, r] \leftarrow \text{temp}$ ;
16:        end if
17:      end for
18:    end for
19:  end for
20:   $\vec{L} \leftarrow d[0, k]$ ;
21: end while

```

the inner loop determines the optimal solution for merging jobs within that range by subproblems.

When k jobs are dispatched simultaneously, the time complexity of initializing the dynamic programming array d in Algorithm 3 is $O(k)$. Assuming the time complexity of the *func* function is $O(\varphi(m))$, the time complexity of the dynamic programming process is $O(\varphi(m) \cdot k^3)$, making the total time complexity $O(\varphi(m) \cdot k^3)$. Additionally, maintaining the dynamic programming array incurs a space complexity of $O(m \cdot k^2)$.

V. EVALUATION

While our model is constructed under relatively strong assumptions, this deliberate design choice allows us to distill and rigorously examine the fundamental theoretical characteristics of the batch-ordered job-store scheduling problem. Such abstraction not only facilitates a deeper understanding of the problem's intrinsic complexity but also enables the derivation of generalizable performance bounds that transcend specific datasets. Our proposed algorithms are grounded in these theoretical insights, offering principled and practically relevant approaches that can inspire robust scheduling strategies in real-world systems.

Although heuristic or learning-based methods may appear more adaptive in dynamic environments, they often lack theoretical guarantees and are not readily applicable to our model without significant adaptation. Consequently, they do not serve as suitable baseline algorithms for evaluating our contributions within this framework.

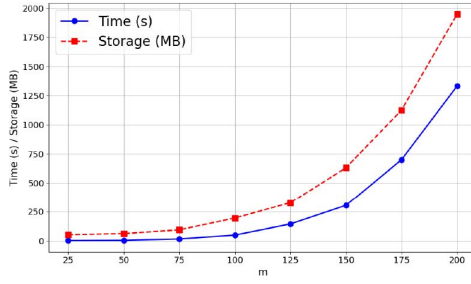


Fig. 6. How DPM's time and space consumption varies with the number of machines m .

The primary objective of the experiments in this section is to validate the theoretical results we have established. We begin by evaluating the DPM algorithm with respect to both time and space efficiency. Subsequently, we conduct simulation experiments to demonstrate the effectiveness of our online algorithm and offline optimization algorithm in achieving balanced load distribution across both scheduling scenarios.

A. DPM Performance

In this experiment, we implemented our algorithms using Python 3 programming language to ensure robust and efficient execution. To measure the performance of DPM algorithm in terms of time and space complexity, we utilized the built-in *time* module to record execution time, and the *memory-profiler* module to monitor memory consumption at 0.01s intervals throughout the execution. These tools provided precise and reliable data on the resource requirements of our implementation. The experiments were conducted on a host machine equipped with an Intel Core i9-10900K CPU at 3.70 GHz, 64 GB of RAM, and running a 64-bit Windows 10 operating system.

The parameters of the DPM algorithm were set as follows: the number of machines, m , increased from 25 to 200, and the lengths of the two circular sequences were set to 60% of the number of machines m . Notably, the closer the length of the circular sequence is to $m/2$, the higher the complexity of the problem, resulting in greater time and storage consumption. The elements within the circular sequences were randomly generated integers ranging from 1 to 10, and the dynamic programming array was stored using integer variables. When the variable type of the dynamic programming array was changed from integers to longer floating-point numbers, both the storage space and computation time consumption will increase accordingly. We recorded the execution time and the maximum memory usage of the DPM algorithm as m varied, as shown in Fig. 6.

To compare the efficiency of DPM algorithm, we also implemented a *Backtracking Pruning-based Matching (BPM)* algorithm [27]: First, we determine a search set for the minimum maximum sum (the maximum value among the elements in the two sequences, as well as the sums of pairs of elements from the two sequences that exceed this maximum value), then use a binary search method to find the achievable minimum maximum sum within this search set. When checking whether a specific value in the search set can be achieved through matching, we

call a backtracking pruning-based method, which performs necessary pruning operations based on constraint conditions during the backtracking process.

The final experimental results show that the time consumption of the BPM algorithm is significantly higher than that of DPM algorithm. Additionally, due to different inputs, the time it takes to find a solution varies greatly, with the worst-case scenario requiring traversal of all nodes in the search tree. When $m = 25$, we ran BPM algorithm ten times with random inputs; its fastest execution time was 45.14 seconds, the slowest was 947.81 seconds, and the average execution time was 375.99 seconds. Although the DPM algorithm requires maintaining a large dynamic programming array, which consumes more memory, it takes less than a second to solve the problem of size $m = 25$. For larger problems, BPM algorithm becomes almost unmanageable. However, as the problem size further increases, obtaining an exact solution using the DPM algorithm also becomes increasingly time- and memory-consuming. In such cases, it would be beneficial to explore approximation algorithms that build upon the DPM framework and trade a marginal loss in accuracy for improved efficiency. A detailed investigation of such algorithms is beyond the scope of this study as discussed in Section IV-A.

B. Load Balancing

This subsection aims to evaluate the practical performance of our proposed online and offline scheduling algorithms for load balancing. It is important to note that the experimental results primarily serve to validate the theoretical model; the actual performance does not influence the theoretical properties or conclusions derived from our analysis.

We assess performance using two key metrics. First, we compute the standard deviation of machine loads after scheduling, which directly reflects the resulting load balance. Second, we measure the ratio between the maximum load and the average load, providing an intuitive indicator of the convergence behavior across different algorithms. For this ratio, we first calculate the average load per machine, i.e., the value that would result if all loads were perfectly and evenly distributed across all machines, which is typically impossible for any algorithm to achieve. At any moment τ , the average load is given by:

$$L_{avg} = \frac{S}{m}. \quad (30)$$

Since the average load L_{avg} is less than or equal to the optimal load L^* , we have:

$$r = \frac{L}{L_{avg}} \geq \frac{L}{L^*}, \quad (31)$$

here, the right-hand side of the inequality represents the expression of the competitive ratio.

Meanwhile, we introduce a baseline called the *Randomized Load Minimization (RLM)* algorithm [28]: The RLM algorithm generates several random scheduling schemes (10 in our experiments) for each job, ensuring all requirements are met. It then selects the scheme that minimizes the maximum load across the machines.

TABLE III
NORMAL DISTRIBUTIONS BASED ON VEHICLE ATTRIBUTES

Vehicle Type	Length of Vehicle (Feet)	Normal Distribution
Motorcycle	Any	$\mathcal{N}(12, 7^2)$
Auto	≤ 15	$\mathcal{N}(20, 8.5^2)$
Auto	> 15	$\mathcal{N}(25, 10^2)$
Truck	Any	$\mathcal{N}(30, 12^2)$

We take vehicle service at toll plazas as a representative example. This scenario is selected due to its consistency with the underlying job-store model and the availability of abundant real-world traffic data [21]. Specifically, vehicles arriving from multiple lanes within the same time window can be regarded as a job [26]. Assigning them to distinct toll booths (or further preserving the left-to-right lane order) helps reduce path conflicts and enhances both safety and efficiency.

We utilized the *Next Generation Simulation* (NGSIM) dataset [21], based on traffic data from two subsets, US-101 and I-80, which recorded highway segments. At the same recording point, we generated vehicle batches (jobs) using a 1-second time window (or when a second vehicle appeared in the same lane). Since US-101 and I-80 contained 5 and 6 main lanes respectively, to better demonstrate the effect of the algorithm, we extracted 200 consecutive vehicle batches and divided them into two segments, then merged them into 100 vehicle batches containing 10 and 12 main lanes.

In the simulation setup, we determined the service time for each vehicle at the toll booth based on distribution patterns derived from existing record data, applying different normal distributions according to the vehicle type and length. The exact normal distributions used are presented in Table III. The service times were constrained to fall within a range of 3 to 60 seconds.

We set the job size for DPO's optimization process to 10. Figs. 7 and 8 show the load standard deviation across machines for each algorithm in the exclusive and circular scenarios, respectively, while Figs. 9 and 10 present the ratio of the maximum load to the average load for each algorithm in the exclusive and circular scenarios, respectively.

Since the RLM algorithm is non-deterministic, its results vary with each execution. Nevertheless, we observe that the LG and OLG algorithms consistently outperform the RLM algorithm, typically leading to better load balancing and smaller maximum loads. Meanwhile, the DPO algorithm, which incorporates offline optimization, achieves the best results.

These findings indicate that the RLM algorithm not only performs worse overall compared to the proposed algorithms but also exhibits lower stability. Due to the vast solution space, the RLM algorithm may experience significant fluctuations across multiple runs. As the number of machines m increases, the solution space expands, further degrading RLM's load balancing performance. In contrast, our proposed algorithms effectively mitigate this limitation, demonstrating strong potential for both future research and practical deployment optimization.

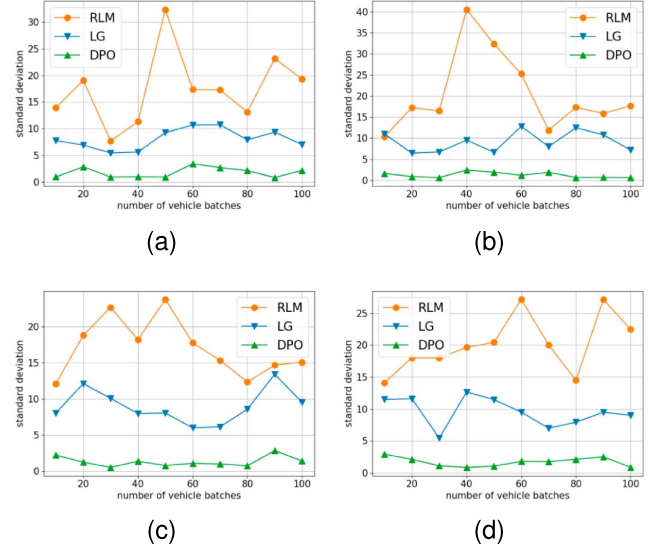


Fig. 7. Variation in standard deviation based on traffic data in the exclusive scenario. (a) US-101: 7:50 a.m. to 8:05 a.m., representing the buildup of congestion. (b) US-101: 8:20 a.m. to 8:35 a.m., representing full congestion during the peak period. (c) I-80: 4:00 p.m. to 4:15 p.m., representing the buildup of congestion. (d) I-80: 5:15 p.m. to 5:30 p.m., representing full congestion during the peak period.

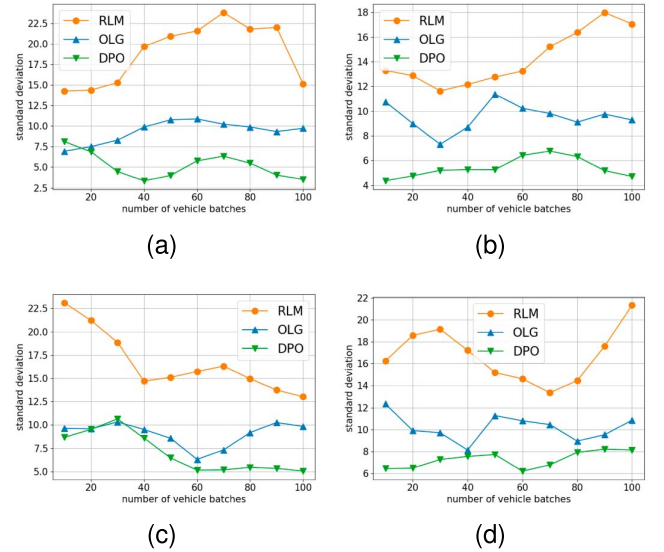


Fig. 8. Variation in standard deviation based on traffic data in the circular scenario. (a) US-101: 7:50 a.m. to 8:05 a.m., representing the buildup of congestion. (b) US-101: 8:20 a.m. to 8:35 a.m., representing full congestion during the peak period. (c) I-80: 4:00 p.m. to 4:15 p.m., representing the buildup of congestion. (d) I-80: 5:15 p.m. to 5:30 p.m., representing full congestion during the peak period.

VI. RELATED WORK

Task scheduling under various constraints is a fundamental topic in computer science and operations research due to its broad applicability in real-world systems [1], [2], [3], [4]. In this section, we review scheduling problems that are similar to or related to the job-store model.

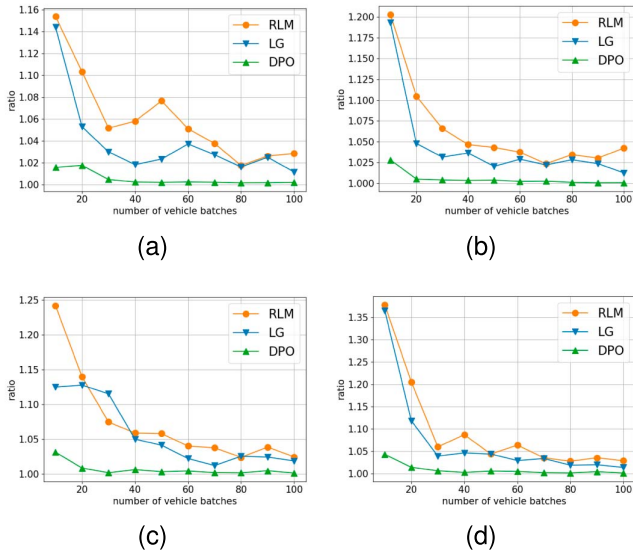


Fig. 9. Variation in the ratio of maximum load to average load based on traffic data in the exclusive scenario. (a) US-101: 7:50 a.m. to 8:05 a.m., representing the buildup of congestion. (b) US-101: 8:20 a.m. to 8:35 a.m., representing full congestion during the peak period. (c) I-80: 4:00 p.m. to 4:15 p.m., representing the buildup of congestion. (d) I-80: 5:15 p.m. to 5:30 p.m., representing full congestion during the peak period.

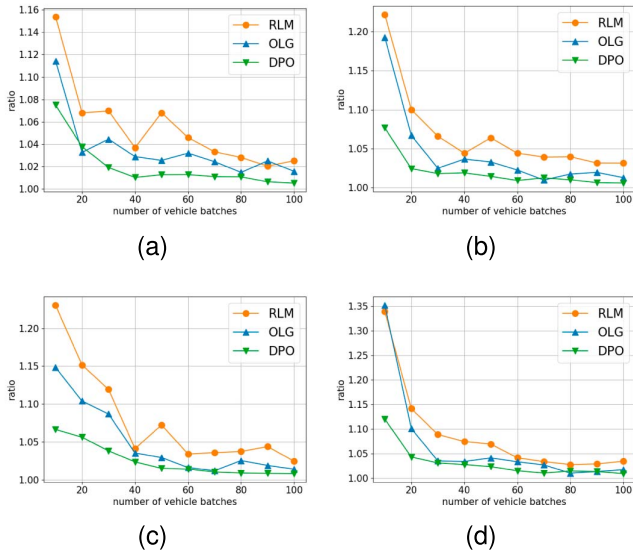


Fig. 10. Variation in the ratio of maximum load to average load based on traffic data in the circular scenario. (a) US-101: 7:50 a.m. to 8:05 a.m., representing the buildup of congestion. (b) US-101: 8:20 a.m. to 8:35 a.m., representing full congestion during the peak period. (c) I-80: 4:00 p.m. to 4:15 p.m., representing the buildup of congestion. (d) I-80: 5:15 p.m. to 5:30 p.m., representing full congestion during the peak period.

The Job-Shop Scheduling Problem [5] is a classical model for scheduling a set of jobs on multiple machines. Each job comprises a strictly ordered sequence of operations, where each operation must be processed on a designated machine, and no operation can begin until its predecessor has finished. JSSP is NP-hard in the general case [23], and extensive research has proposed numerous algorithmic solutions [29], [30]. Several

important variants have been developed to address real-world constraints. The Flexible Job-Shop Scheduling Problem [31] introduces flexibility by allowing each operation to be processed on one of several eligible machines. The Dynamic Job-Shop Scheduling Problem [32] considers real-time disruptions such as machine breakdowns or unexpected job arrivals. The Blocking Job-Shop Scheduling Problem [17] models scenarios with limited or no intermediate storage, requiring continuous operation flow. The Robust Job-Shop Scheduling Problem [18] aims to ensure schedule stability under processing time variability or uncertainty.

Parallel machine scheduling [8] focuses on assigning tasks to multiple machines to optimize objectives such as makespan, total completion time, or tardiness. It is widely applicable in parallel computing and industrial manufacturing, where tasks can be executed concurrently. Challenges arise particularly when machines are heterogeneous or when precedence constraints exist [19]. Despite its simple structure, the problem remains NP-hard, and many practical heuristics and approximation algorithms have been developed to address it efficiently.

Flow shop scheduling [9] is another classical scheduling problem in which all jobs are processed on machines in the same fixed order. It shares similarities with pipeline task scheduling, where tasks also follow a strict stage-wise progression. However, while flow shop scheduling emphasizes optimizing job-level performance metrics, pipeline scheduling focuses more on maintaining continuous task flow and avoiding stage-level bottlenecks [20]. The latter is prevalent in domains such as data streaming, network transmission, and hardware pipelines.

Round-robin scheduling is a widely used method, particularly in CPU scheduling and distributed systems [33], [34], focusing on fair and balanced distribution of tasks. However, classical round-robin typically operates at the level of independent tasks or processes assigned to a single unit. In contrast, the round-robin process in job-store scheduling deals with a collection of interdependent tasks, subject to structural constraints like exclusivity or circular ordering. Under this global round-robin assignment of job operations to machines, our model further enforces balanced load constraints, which introduces additional structural complexity not present in classical round-robin models.

In summary, our work explores a novel scheduling paradigm that integrates batch-ordered jobs, interdependent operations, and global round-robin allocation under load balancing constraints. Unlike classical models, our scenario necessitates a combination of structure-aware scheduling and theoretically grounded algorithm design. Therefore, we propose rule-based algorithms with provable guarantees, tailored to the unique characteristics of job-store scheduling.

VII. CONCLUSION

In this paper, we introduce a novel batch-ordered job-store scheduling model and conduct in-depth analyses under two distinct scenarios, each with corresponding scheduling strategies. In the online setting, we focus on analyzing the tight competitive ratio of $2 - \frac{1}{m}$ for the greedy algorithm under the exclusive

scenario, and investigate potential maintaining order anomalies that may arise when achieving locally optimal scheduling in the circular scenario. In the offline setting, we model the locally optimal scheduling problem in the circular scenario as a circular sequence alignment problem and propose an efficient solution. Furthermore, we explore optimal merging orders for scheduling multiple jobs simultaneously, aiming to enhance overall load balancing.

The model abstracts only the essential elements of real-world scheduling situations, allowing us to explore the inherent complexity of the problem and establish relevant theoretical bounds. Therefore, the primary contribution of this work lies not in modeling a comprehensive application scenario with all practical variables, but in laying a general theoretical foundation for the batch-ordered job-store problem. These theoretical insights serve as a cornerstone for future efforts to design practical scheduling algorithms that can incorporate real-world uncertainties.

REFERENCES

- [1] D. Narayanan, A. Phanishayee, K. Shi, X. Chen, and M. Zaharia, "Memory-efficient pipeline-parallel DNN training," in *Proc. 38th Int. Conf. Mach. Learn.*, vol. 139, Jul. 18–24, 2021, pp. 7937–7947.
- [2] T. K. Agrawal, A. Sahu, M. Ghose, and R. Sharma, "Scheduling chained multiprocessor tasks onto large multiprocessor system," *Computing*, vol. 99, pp. 1007–1028, Oct. 2017.
- [3] F. Wang, S. Oral, G. Shipman, O. Drokun, T. Wang, and I. Huang, "Understanding lustre filesystem internals," Oak Ridge National Laboratory, National Center for Computational Sciences, Tech. Rep. 2009. [Online]. Available: https://wiki.old.lustre.org/images/d/da/Understanding_Lustre_Filesystem_Internals.pdf
- [4] Y. Lin et al., "QUART: Latency-aware FaaS system for pipelining large model inference," in *Proc. IEEE 44th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, 2024, pp. 1–12.
- [5] H. Xiong, S. Shi, D. Ren, and J. Hu, "A survey of job shop scheduling problem: The types and models," *Comput. Oper. Res.*, vol. 142, Jun. 2022, Art. no. 105731.
- [6] C. Menard et al., "High-performance deterministic concurrency using Lingua Franca," *ACM Trans. Archit. Code Optim.*, vol. 20, no. 4, Oct. 2023, pp. 1–29.
- [7] W. Shi, S. Zhou, Z. Niu, M. Jiang, and L. Geng, "Multiuser co-inference with batch processing capable edge server," *IEEE Trans. Wireless Commun.*, vol. 22, no. 1, pp. 286–300, Jan. 2023.
- [8] K.-C. Ying, P. Pourhejazy, and X.-Y. Huang, "Revisiting the development trajectory of parallel machine scheduling," *Comput. Oper. Res.*, vol. 168, Aug. 2024, Art. no. 106709.
- [9] J. S. Neufeld, S. Schulz, and U. Buscher, "A systematic review of multi-objective hybrid flow shop scheduling," *Eur. J. Oper. Res.*, vol. 309, no. 1, pp. 1–23, 2023.
- [10] A. Kulik, M. Mnich, and H. Shachnai, "Improved approximations for vector bin packing via iterative randomized rounding," in *Proc. IEEE 64th Annu. Symp. Found. Comput. Sci. (FOCS)*, 2023, pp. 1366–1376.
- [11] N. Ueter, M. Günzel, G. v d Brüggen, and J.-J. Chen, "Parallel path progression DAG scheduling," *IEEE Trans. Comput.*, vol. 72, no. 10, pp. 3002–3016, Oct. 2023.
- [12] A. Tusha and H. Arslan, "Interference burden in wireless communications: A comprehensive survey from PHY layer perspective," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 4, pp. 2204–2246, Aug. 2024.
- [13] M. Jonsson, B. Svensson, M. Taveniku, and A. Ahlander, "Fiber-ribbon pipeline ring network for high-performance distributed computing systems," in *Proc. Int. Symp. Parallel Archit., Algorithms Netw. (I-SPAN)*, 1997, pp. 138–143.
- [14] X. Zhu et al., "On optimizing traffic scheduling for multi-replica containerized microservices," in *Proc. 52nd Int. Conf. Parallel Process. (ICPP)*, New York, NY, USA: ACM, 2023, pp. 358–368.
- [15] Y. Mao, X. Shang, and Y. Yang, "Ant colony based online learning algorithm for service function chain deployment," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, 2023, pp. 1–10.
- [16] M. F. Faraj and C. Schulz, "Buffered streaming graph partitioning," *ACM J. Exp. Algorithmics*, vol. 27, pp. 1–26, Oct. 2022.
- [17] J. K. Mogali, L. Barbulescu, and S. F. Smith, "Efficient primal heuristic updates for the blocking job shop problem," *Eur. J. Oper. Res.*, vol. 295, no. 1, pp. 82–101, 2021.
- [18] R. L. C. Souza, A. Ghasemi, A. Saif, and A. Gharaei, "Robust job-shop scheduling under deterministic and stochastic unavailability constraints due to preventive and corrective maintenance," *Comput. Ind. Eng.*, vol. 168, Jun. 2022, Art. no. 108130.
- [19] V. Fagnon, G. Lucarelli, and C. Rapine, "Makespan minimization for scheduling on heterogeneous platforms with precedence constraints," in *Proc. Euro-Par, Parallel Process.*, Cham, Switzerland: Springer Nature Switzerland, 2024, pp. 343–356.
- [20] Z. Zheng, X. Ren, F. Xue, Y. Luo, X. Jiang, and Y. You, "Response length perception and sequence scheduling: An LLM-empowered LLM inference pipeline," in *Proc. Adv. Neural Inf. Process. Syst.*, Red Hook, NY, USA: Curran Associates, 2023, vol. 36, pp. 65517–65530.
- [21] U.S. Department of Transportation Federal Highway Administration. 2016, "Next generation simulation (NGSIM) vehicle trajectories and supporting data," Dataset, provided by ITS DataHub through Data.transportation.gov. Accessed 20240701 from, doi: <http://doi.org/10.21949/1504477>.
- [22] R. M. Karp, *Reducibility Among Combinatorial Problems*. Berlin, Germany: Springer Berlin Heidelberg, 2010, pp. 219–241.
- [23] M. R. Garey and D. S. Johnson, *Computers and Intractability*, vol. 174. San Francisco, CA, USA: Freeman, 1979.
- [24] S. Jukna, *The Pigeonhole Principle*. Berlin, Germany: Springer Berlin Heidelberg, 2011, pp. 53–75.
- [25] R. L. Graham, "Bounds for certain multiprocessing anomalies," *Bell Syst. Tech. J.*, vol. 45, no. 9, pp. 1563–1581, 1966.
- [26] M. Zhou, B. Zhao, M. Xu, H. Dai, and Y. Wang, "TollHelper: A safe and efficient traffic control approach on toll plaza via constrained load balancing," in *Proc. IEEE Int. Symp. Parallel Distrib. Process. Appl. (ISPA)*, 2024, pp. 378–385.
- [27] P. van Beek, "Chapter 4 - Backtracking search algorithms," in *Handbook of Constraint Programming (Foundations of Artificial Intelligence)*, vol. 2. New York, NY, USA: Elsevier, 2006, vol. 2, pp. 85–134.
- [28] Y. Nakatsukasa and J. A. Tropp, "Fast and accurate randomized algorithms for linear systems and eigenvalue problems," *SIAM J. Matrix Anal. Appl.*, vol. 45, no. 2, pp. 1183–1214, 2024.
- [29] J. Carlier and É. Pinson, "An algorithm for solving the job-shop problem," *Manage. Sci.*, vol. 35, no. 2, pp. 164–176, 1989.
- [30] D. B. Fontes, S. M. Homayouni, and J. F. Gonçalves, "A hybrid particle swarm optimization and simulated annealing algorithm for the job shop scheduling problem with transport resources," *Eur. J. Oper. Res.*, vol. 306, no. 3, pp. 1140–1157, 2023.
- [31] S. Dauzère-Pères, J. Ding, L. Shen, and K. Tamssaouet, "The flexible job shop scheduling problem: A review," *Eur. J. Oper. Res.*, vol. 314, no. 2, pp. 409–432, 2024.
- [32] J. Mohan, K. Lanka, and A. N. Rao, "A review of dynamic job shop scheduling techniques," *Procedia Manuf.*, vol. 30, pp. 34–39, Feb. 2019.
- [33] C. Sharma, S. Sharma, S. Kautish, S. A. M. Alsallami, E. Khalil, and A. Wagdy Mohamed, "A new median-average round robin scheduling algorithm: An optimal approach for reducing turnaround and waiting time," *Alexandria Eng. J.*, vol. 61, no. 12, pp. 10527–10538, 2022.
- [34] S. Fan, D. Yue, H. Yan, and C. Deng, "Distributed round-robin protocol-based secondary control for ESS under FDI attacks," *IEEE Trans. Ind. Electron.*, vol. 71, no. 12, pp. 16493–16502, Dec. 2024.



Mengbing Zhou received the bachelor's degree in information and computing science from the South China University of Technology, in 2022. He is currently working toward the master's degree in computer technology with the University of Chinese Academy of Sciences (Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences). His research interests include cloud computing and big data analytics.



ences, as a Professor. His research interests include cloud computing, big data analytics, and Java virtual machines on multicores.



Yang Wang (Member, IEEE) received the B.Sc. degree in applied mathematics from the Ocean University of China, Qingdao, China, in 1989, the M.Sc. degree in computer science from Carleton University, Ottawa, Canada, in 2001, and the Ph.D. degree in computer science from the University of Alberta, Edmonton, Canada, in 2008. From 2009 to 2011, he was an Alberta Industry R&D Associate, and a Canadian Fulbright Scholar, from 2014 to 2015. He currently works with Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences,

Bocong Zhao (Graduate Student Member, IEEE) received the B.Sc. degree in e-commerce engineering with law from Beijing University of Posts and Telecommunications, in 2018, and the M.Sc. degree in information science from the University of Pittsburgh, USA, in 2021. He is currently working toward the Ph.D. degree with the University of Chinese Academy of Sciences (Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences). His research interests including distributed computing and cloud computing.



Chengzhong Xu (Fellow, IEEE) received the Ph.D. degree from the University of Hong Kong, in 1993. He is currently the Dean of Faculty of Science and Technology, University of Macau, China, and the Director of the Institute of Advanced Computing and Data Engineering, Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences. He has published more than 200 papers in journals and conferences. His research interests include parallel and distributed systems, service and cloud computing, and software engineering. He serves on a number of journal editorial boards, including IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON CLOUD COMPUTING, *Journal of Parallel and Distributed Computing* and *China Science Information Sciences*.